{EPITECH}

CUDDLE_

< CODE YOUR OWN PANDA, BECAUSE EVEN DATA NEEDS A CUDDLE />

CUDDLE

Preliminaries

binary name: libcuddle.a
language: C
compilation: via Makefile, including re, clean and fclean rules
Authorized functions: all of the libC and libmath functions are authorized.

- ✓ The totality of your source files, except all useless files (binary, temp files, objfiles,...), must be included in your delivery.
- ✓ All the bonus files (including a potential specific Makefile) should be in a directory named bonus.
- ✓ Error messages have to be written on the error output, and the program should then exit with the 84 error code (0 if there is no error).

The Cuddle project aims to re-code the popular pandas library, a cornerstone tool in the data manipulation ecosystem, especially for handling CSV files and complex data structures.

Pandas is renowned for its robust capabilities in data cleaning, transformation, and analysis, offering a user-friendly interface that makes data science more accessible and efficient.

We highly recommend that you explore the Python pandas library to get a practical sense of its functionalities before diving into the recoding process.

Getting started

For this project, the input file will be a CSV (Comma Separated Values) file, which is a common file format for storing tabular data. The CSV file will contain a header row with the column names and subsequent rows with the data values.

Here is an example of a CSV file:

```
name,age,city
Alice,25,Paris
Bob,30,London
Charlie,35,Berlin
```

The goal of the "My Panda" project is to create a program that can:

- ✓ read a CSV file,
- ✓ store the data in a custom data structure,
- ✓ perform various operations on the data, such as:
  - filtering,
  - sorting,
  - aggregating.

Your program should be able to handle CSV files with different column types (e.g., strings, integers, floats) and provide a flexible interface for interacting with the data. This interface will be a static library (**libcuddle.a**) that can be linked to other programs for data analysis.

Think about the data structure you will use to store the CSV data and how you will implement the various operations efficiently. linked lists, arrays, ... ?

Core functions

The first step is to create a function that reads a CSV file and stores the data in the custom data structure. You can start by defining the data structure `dataframe_t` and the function `df_read_csv` that reads the CSV file and populates the `dataframe_t` structure.

You must define the `dataframe_t` structure in a header file **include/dataframe.h**.

```
typedef struct dataframe_s {
    int nb_rows;
    int nb_columns;
    // ...
} dataframe_t;
```

```
dataframe_t *df_read_csv(const char *filename, const char *separator);
```

Your program must be able to read a CSV file with a custom separator (e.g., comma, semicolon, tab). The default separator if separator is NULL is the comma.

If you need some help and/or feel lost, maybe you should check the Bootstrap.

The `df_write_csv` function should write the data from the `dataframe_t` structure to a CSV file and return 0 if successful. Using the same separator as the input file.

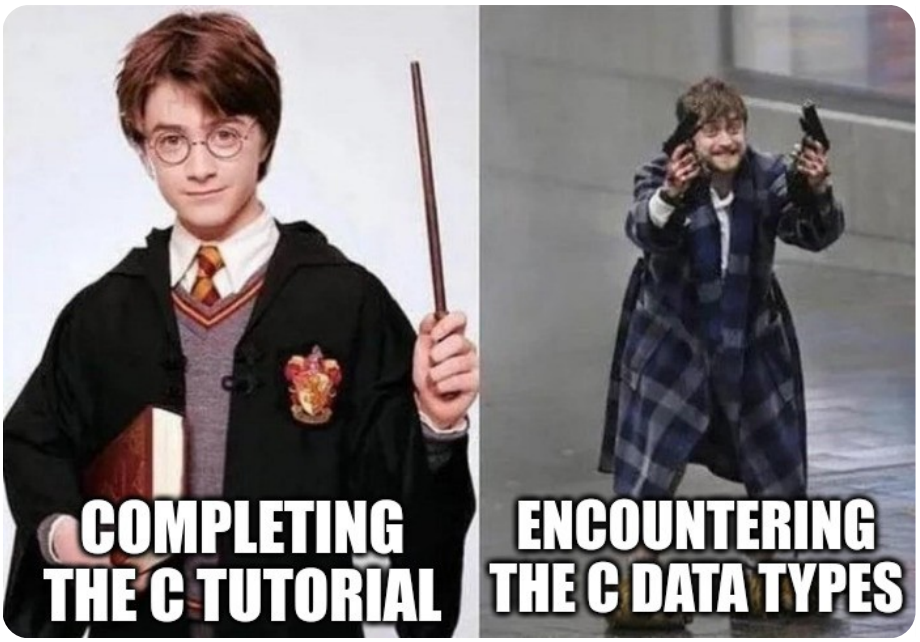
```
int df_write_csv(dataframe_t *dataframe, const char *filename);
```

You must handle the following data types:

```
typedef enum {
    BOOL
    INT,
    UINT,
    FLOAT,
    STRING,
    UNDEFINED // only used internally before the type is determined
} column_type_t;
```

- ✓ String: is the default type if the column contains mixed types.
- ✓ Bool: value can only be ”true” or ”false” (case insensitive).
- ✓ Int: if there is at least one negative value otherwise it's an unsigned int.
- ✓ Float: if there is at least one float value.

⚠️ A value is considered as a number only if it contains only digits and a single dot (.) for float values. For instance, ”-25” is an integer, ”25.0” is a float, ”25.0.0” or ”10 000” are strings.



Basics operations

Once you have the core functions implemented, you can start adding basic operations to manipulate the data in the `dataframe_t` structure.

Terminal

```
$> cat data.csv
name,age,city
Alice,25,Paris
Bob,30,London
Charlie,35,Berlin
Léo,25,Paris
Nathan,30,London
Alex,35,Berlin
Paul,25,Paris
```

Every function should returned totally **independent** dataframe.
If an error occurs, the function should return **NULL**.

head

The `df_head` function should return a new dataframe representing the first `n` rows of the given dataframe.

```c
dataframe_t *df_head(dataframe_t *dataframe, int nb_rows);

int main(void) {
    dataframe_t *dataframe = df_read_csv("data.csv", NULL);
    dataframe_t *head = df_head(dataframe, 3);

    df_write_csv(head, "head.csv");
    return 0;
}
```

Terminal

```
$> ./cuddle && cat head.csv
name,age,city
Alice,25,Paris
Bob,30,London
Charlie,35,Berlin
```

tail

The `df_tail` function should return a new dataframe representing the last `n` rows of the given dataframe.

```
dataframe_t *df_tail(dataframe_t *dataframe, int nb_rows);

int main(void) {
    dataframe_t *dataframe = df_read_csv("data.csv", NULL);
    dataframe_t *tail = df_tail(dataframe, 3);

    df_write_csv(tail, "tail.csv");
    return 0;
}
```

Terminal

```
$> ./cuddle && cat tail.csv
name,age,city
Nathan,30,London
Alex,35,Berlin
Paul,25,Paris
```

shape

The `df_shape` function should print the number of rows and columns in the dataframe using a dataframe_shape_t structure that contains two fields: `nb_rows` and `nb_columns`.

```
typedef struct dataframe_shape_s {
    int nb_rows;
    int nb_columns;
} dataframe_shape_t;

dataframe_shape_t df_shape(dataframe_t *dataframe);

int main(void) {
    dataframe_t *dataframe = df_read_csv("data.csv", NULL);
    dataframe_shape_t shape = df_shape(dataframe);

    printf("Shape: %d rows, %d columns\n", shape.nb_rows, shape.nb_columns);
    return 0;
}
```

Terminal

```
$> ./cuddle
Shape: 7 rows, 3 columns
```

info

The `df_info` function should print information about the dataframe, including the column names and types.

```
void df_info(dataframe_t *dataframe);

int main(void) {
    dataframe_t *dataframe = df_read_csv("data.csv", NULL);
    df_info(dataframe);
    return 0;
}
```

Terminal

```
$> ./cuddle
3 columns:
- name: string
- age: unsigned int
- city: string
```

Types should be displayed in lowercase: bool, int, unsigned int, float, string.

describe

The `df_describe` function should provide summary statistics for the numerical columns in the dataframe, including count, mean, standard deviation, minimum and maximum.

```
void df_describe(dataframe_t *dataframe);

int main(void) {
    dataframe_t *dataframe = df_read_csv("data.csv", NULL);
    df_describe(dataframe);
    return 0;
}
```

Terminal

```
$> ./cuddle
Column: age
Count: 7
Mean: 29.29
Std: 4.16
Min: 25.00
Max: 35.00
```

Filtering

The next step is to add filtering capabilities to the dataframe. The `df_filter` function should return a new dataframe that contains only the rows that satisfy a specified condition.

```
dataframe_t *df_filter(dataframe_t *dataframe, const char *column, bool (*filter_func)(void
*value));


bool filter_func(void *value) {
    return *(int *)value > 30;
}

int main(void) {
    dataframe_t *dataframe = df_read_csv("data.csv", NULL);
    dataframe_t *filtered = df_filter(dataframe, "age", filter_func);

    df_write_csv(filtered, "filtered.csv");
    return 0;
}
```

Terminal

```
$> ./cuddle && cat filtered.csv
name,age,city
Charlie,35,Berlin
Alex,35,Berlin
```

Sorting

The `df_sort` function should return a new dataframe that contains the same data as the original dataframe but sorted based on a specified column.

```
dataframe_t *df_sort(dataframe_t *dataframe, const char *column, bool (*sort_func)(void *value1,
void *value2));

bool sort_func(void *value1, void *value2) {
    return *(int *)value1 > *(int *)value2;
}

int main(void) {
    dataframe_t *dataframe = df_read_csv("data.csv", NULL);
    dataframe_t *sorted = df_sort(dataframe, "age", sort_func);

    df_write_csv(sorted, "sorted.csv");
    return 0;
}
```

Terminal

```
$> ./cuddle && cat sorted.csv
name,age,city
Alice,25,Paris
Léo,25,Paris
Paul,25,Paris
Bob,30,London
Nathan,30,London
Charlie,35,Berlin
Alex,35,Berlin
```

Aggregation

The `df_groupby` function should return a new dataframe that groups the rows of the dataframe based on a column name and aggregate the values of other columns using an aggregation function.

```
dataframe_t *df_groupby(dataframe_t *dataframe, const char *aggregate_by, const char
**to_aggregate, void *(*agg_func)(void **values, int nb_values));

void *agg_func(void **values, int nb_values) {
    int *sum = malloc(sizeof(int));
    *sum = 0;

    for (int i = 0; i < nb_values; i++) {
        *sum += *(int *)values[i];
    }
    return sum;
}

int main(void) {
    dataframe_t *dataframe = df_read_csv("data.csv", NULL);
    dataframe_t *grouped = df_groupby(dataframe, "city", (const char *[]){"age", NULL},
agg_func);

    df_write_csv(grouped, "grouped.csv");
    return 0;
}
```

Terminal

```
$> ./cuddle && cat grouped.csv
city,age
Paris,75
London,60
Berlin,70
```

Transformation

apply

The `df_apply` function should return a new dataframe with a column transformed by an apply function that takes a value from the column and returns a new value.

```c
dataframe_t *df_apply(dataframe_t *dataframe, const char *column, void *(*apply_func)(void
*value));

void *apply_func(void *value) {
    int *new_value = malloc(sizeof(int));
    *new_value = *(int *)value * 2;
    return new_value;
}

int main(void) {
    dataframe_t *dataframe = df_read_csv("data.csv", NULL);
    dataframe_t *applied = df_apply(dataframe, "age", apply_func);

    df_write_csv(applied, "applied.csv");
    return 0;
}
```

Terminal

```
$> ./cuddle && cat applied.csv
name,age,city
Alice,50,Paris
Bob,60,London
Charlie,70,Berlin
Léo,50,Paris
Nathan,60,London
Alex,70,Berlin
Paul,50,Paris
```

to_type

The `df_to_type` function should return a new dataframe with a column converted to another type based on downcast parameter.

```
dataframe_t *df_to_type(dataframe_t *dataframe, const char *column, column_type_t downcast);

void *apply_func(void *value) {
    char *str = (char *)value;
    if (str[strlen(str) - 1] == 'e')
        str[strlen(str) - 1] = '\0';
    return str;
}

int main(void) {
    dataframe_t *dataframe = df_read_csv("money.csv", NULL);
    dataframe = df_apply(dataframe, "amount", apply_func);
    dataframe_t *new_dataframe = df_to_type(dataframe, "amount", INT);
    df_info(new_dataframe);
    df_write_csv(new_dataframe, "numeric.csv");
    return 0;
}
```

Terminal

```
$> cat money.csv
name,amount
Alice,25e
Bob,30e
Léo,25e
./cuddle && cat numeric.csv
2 columns:
- name: string
- amount: int
name,amount
Alice,25
Bob,30
Léo,25
```

The downcast parameter can be any type of column_type_t.
You must handle the case where the conversion is not possible (e.g., ”hello” to int or ”-25” to uint) by returning a NULL dataframe.

The float to int conversion should truncate the decimal part.
When converting to bool, any non-zero value should be considered as true.

Utilities

get_value

The `df_get_value` function should return the value at the specified row and column index in the dataframe.

```c
void *df_get_value(dataframe_t *dataframe, int row, const char *column);

int main(void) {
    dataframe_t *dataframe = df_read_csv("data.csv", NULL);
    void *value = df_get_value(dataframe, 0, "age");
    printf("Value: %d\n", *(int *)value);
    return 0;
}
```

Terminal

```
$> ./cuddle
Value: 25
```

get_values

The `df_get_values` function should return an array of values in the specified column of the dataframe.

```c
void **df_get_values(dataframe_t *dataframe, const char *column);

int main(void) {
    dataframe_t *dataframe = df_read_csv("data.csv", NULL);
    void **values = df_get_values(dataframe, "age");

    while (*values) {
        printf("Value: %d\n", *(int *)*values);
        values++;
    }
}
```

```
Terminal
$> ./cuddle
Value: 25
Value: 30
Value: 35
Value: 25
Value: 30
Value: 35
Value: 25
```

get_unique_values

The `df_get_unique_values` function should return an array of unique values in the specified column of the dataframe.

```c
void **df_get_unique_values(dataframe_t *dataframe, const char *column);

int main(void) {
    dataframe_t *dataframe = df_read_csv("data.csv", NULL);
    void **values = df_get_unique_values(dataframe, "city");

    while (*values) {
        printf("Value: %s\n", (char *)*values);
        values++;
    }
    return 0;
}
```

```
Terminal
$> ./cuddle
Value: Paris
Value: London
Value: Berlin
```

free

The `df_free` function should free the memory allocated for the dataframe and its components.

```c
void df_free(dataframe_t *dataframe);

int main(void) {
    dataframe_t *dataframe = df_read_csv("data.csv", NULL);

    df_free(dataframe);
    return 0;
}
```

Be careful to implement first the df_read_csv and df_write_csv functions that will be use to test your project.

Bonus

Here are some additional features you can implement to enhance the functionality of your library:

- ✓ **Join**:
  Implement a function to join two dataframes based on a common column.
- ✓ **Performance optimization**:
  Optimize the performance of the library by using efficient data structures and algorithms.
- ✓ **Error handling**:
  Add error handling to the library to provide informative messages when an operation fails.
- ✓ **Export to other formats**:
  Add support to export the dataframe to other formats such as JSON, Excel, or SQL databases.
- ✓ **Interactive interface**:
  Create an interactive CLI for users to interact with the dataframe and perform operations.
- ✓ **Documentation**:
  Document your library, its functions, data structures, and usage examples.
- ✓ ...

Conclusion

The Cuddle project provides an opportunity to develop your programming skills and deepen your understanding of data manipulation techniques. By re-coding the pandas library in C, you will gain valuable experience in handling tabular data, implementing data structures, and designing efficient algorithms.

We encourage you to explore the full range of pandas functionalities and experiment with different data sets to test the robustness and flexibility of your implementation. You can also compare your results with the pandas library to ensure that your version produces accurate and consistent output.

Unit tests are essential to verify the correctness of your implementation and ensure that it behaves as expected in various scenarios.
Don't forget to split your code into manageable functions and modules to facilitate testing and debugging.

v 1.0.3

{EPITECH}